

MODULE 3

Object-Oriented Design Using UML

- **Object-oriented design (OOD)** is a software design approach where the system is designed as a collection of interacting objects.
- **UML (Unified Modeling Language)** is commonly used to represent object-oriented designs using diagrams.
- OOD helps identify software components, their relationships, and how they interact.
- A structured object-oriented design process develops different system models.
- Creating and maintaining these models requires significant effort.
- For small systems, detailed modeling may not be cost-effective.
- For large systems, design models are useful for communication between different development teams.
- Common stages in an object-oriented design process:
 - Define the system context and how it will be used.
 - Design the overall system architecture.
 - Identify the main objects in the system.
 - Develop design models.
 - Specify interfaces for objects.
- **System context** describes the relationship between the software and its external environment.
- Understanding the context helps:
 - Decide required functionality.
 - Define system boundaries.
 - Determine which features belong to the system and which belong to external systems.
- **System context model**

- A structural model showing external systems connected to the software.
- **Interaction model**
 - A dynamic model showing how the system interacts with users or other systems during operation.
- After understanding interactions, the system architecture is designed.
- Major components and their interactions are identified.
- Components may be organized using patterns such as **layered architecture** or **client-server architecture**.
- Example: A weather station consists of independent subsystems that communicate through message broadcasting.
- **Object class identification**
 - One of the most difficult parts of object-oriented design.
 - There is no fixed formula for identifying objects.
 - It depends on the designer's experience, skills, and domain knowledge.
 - It is an iterative process and may require several refinements.
- **Approaches to identify objects**
 - Grammatical approach: identify objects from nouns in natural language descriptions.
 - Tangible approach: identify physical entities in the application domain.
 - Behavioural approach: identify objects based on actions and behaviours.
 - Scenario-based approach: identify objects, attributes, and methods from use-case scenarios.
- **Design models**
 - Show objects, object classes, and relationships.
 - Two main types:
 - **Structural models** – describe the static structure of the system.
 - **Dynamic models** – describe interactions and behavior of objects over time.
- **Examples of UML design models**

- Subsystem models – logical grouping of related objects.
 - Sequence models – show the order of interactions between objects.
 - State machine models – show how objects change state in response to events.
 - Other models include use-case, aggregation, and generalization models.
-

Project Planning

- **Project planning** is the process of breaking a project into smaller tasks, assigning them to team members, predicting possible problems, and preparing solutions.
 - A **project plan** is created at the beginning of the project to:
 - Communicate how the work will be done.
 - Guide the project team.
 - Inform customers.
 - Track and assess project progress.
 - **Planning stages (Project Life Cycle):**
 - **Proposal planning**
 - Done while bidding for a software contract.
 - Based on only outline requirements.
 - Used to estimate development cost and decide the project price.
 - **Project startup planning**
 - Done after requirements are better understood.
 - Plans budget, staffing, resource allocation, and monitoring mechanisms.
 - Required even in agile projects for resource planning.
 - **Development planning**
 - Project plan is updated regularly as the project progresses.
 - Schedule, cost estimates, and risks are revised based on new information.
-

Software Pricing

- **Software pricing** is the process of deciding the price to charge a customer for developing a software system.
- It is based on estimating the development cost, but the final price also depends on business and market factors.
- **Cost estimation includes:**
 - Staff costs
 - Hardware costs
 - Software costs
 - Travel expenses
 - Training costs
 - Development effort
- There is **no direct relationship** between development cost and selling price.
- Organizational, economic, political, and business considerations also influence pricing.
- **Factors affecting software pricing:**
 - **Contractual terms**
 - If the developer keeps ownership of the source code for reuse, the price may be lower.
 - If the customer receives full ownership of the source code, the price may be higher.
 - **Cost estimate uncertainty**
 - If cost estimates are uncertain, extra contingency is added to reduce risk.
 - **Financial health**
 - Companies facing financial problems may lower prices to win contracts and maintain cash flow.
 - **Market opportunity**
 - A company may quote a low price to enter a new market or gain future business opportunities.

- **Requirements volatility**
 - If requirements are likely to change, the initial price may be lower, with additional charges applied for future changes.
 - **Pricing strategies:**
 - **Underpricing**
 - Used to win contracts.
 - Helps retain staff by securing work.
 - Can be used to enter a new market.
 - **Increased pricing**
 - Used in fixed-price contracts to cover unexpected risks and uncertainties.
 - **Pricing to win strategy:**
 - The software is priced according to what the customer is willing to pay.
 - If this amount is lower than the development cost:
 - Some features may be postponed to later releases.
 - Future requirement changes may be charged at higher rates to recover costs.
-

Plan-Driven Development

- **Plan-driven development** is a software development approach where the entire development process is planned in detail before implementation.
- It is the traditional way of managing large software projects and is based on engineering project management techniques.
- A **project plan** is created that specifies:
 - Work to be done.
 - Team members responsible for each task.
 - Development schedule.
 - Expected work products.
- Managers use the project plan to:

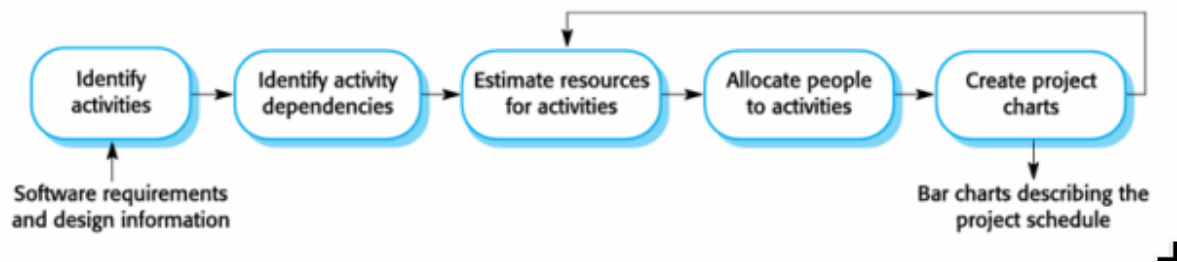
- Support project decision-making.
- Monitor and measure project progress.
- **Advantages of plan-driven development**
 - Early planning helps manage staff availability and resource allocation.
 - Potential problems and task dependencies are identified before development begins.
 - Improves coordination and organization in large projects.
- **Disadvantages of plan-driven development**
 - Many decisions made early may need to be changed later.
 - Changes in requirements or the development environment can make the original plan outdated.
 - Less flexible than agile approaches.
- **Contents of a project plan**
 - Introduction
 - Project organization
 - Risk analysis
 - Hardware and software resource requirements
 - Work breakdown
 - Project schedule
 - Monitoring and reporting mechanisms
- **Supplementary plans**
 - **Configuration management plan:** Defines configuration management procedures and structures.
 - **Deployment plan:** Explains software deployment and data migration.
 - **Maintenance plan:** Estimates future maintenance effort and cost.
 - **Quality plan:** Defines quality standards and procedures.
 - **Validation plan:** Describes the approach, resources, and schedule for system validation.
- **Planning is an iterative process**

- The project plan is updated regularly as development progresses.
 - Requirements, schedules, cost estimates, and risks are revised when new information becomes available.
 - Changes in business goals may also require the project plan to be updated.
-

Project Scheduling

- **Project scheduling** is the process of deciding how project work will be divided into tasks and when and how those tasks will be performed.
- It includes estimating:
 - Time required for each task.
 - Effort needed.
 - People responsible for the task.
 - Resources such as hardware, software, storage, and travel budget.
- **Objectives of project scheduling**
 - Plan the order of tasks.
 - Allocate resources efficiently.
 - Monitor project progress.
 - Complete the project within time and cost limits.
- **Project scheduling activities**
 - Split the project into smaller tasks.
 - Estimate the time and resources needed for each task.
 - Organize tasks to run concurrently where possible for better use of the workforce.
 - Minimize task dependencies to avoid unnecessary delays.
 - Scheduling depends heavily on the project manager's experience and judgment.
- **Detailed scheduling process**
 - Identify project activities from requirements and design.
 - Identify dependencies between activities.

- Estimate effort, cost, and resources for each activity.
- Allocate people based on skills and availability.
- Create project charts to represent and monitor the schedule.
- **Scheduling problems**
 - It is difficult to accurately estimate task difficulty and development cost.
 - Productivity is not directly proportional to the number of people assigned.
 - Adding more people to a delayed project can make it even later due to communication overhead.
 - Unexpected events are common, so contingency should always be included in the schedule.
- **Schedule presentation**
 - Graphical representations are commonly used.
 - **Bar charts (Gantt charts)** show activities or resources against time.
 - **Activity network diagrams** show dependencies between tasks.
 - Tasks should generally be of manageable size (about one or two weeks).
- **Project activities (tasks) should have**
 - A defined duration.
 - An effort estimate (person-days or person-months).
 - A deadline.
 - A clear end point or deliverable, such as a document or completed testing.
- **Milestones and deliverables**
 - **Milestones** are checkpoints used to measure project progress (e.g., system ready for testing).
 - **Deliverables** are work products given to the customer (e.g., requirements document or software component).



Project Duration and Staffing

- **Project Duration** is the total calendar time required to complete a software project.
- **Staffing** refers to the number of personnel required during different phases of software development.
- Project managers estimate both duration and staffing during project planning.
- **COCOMO II Project Duration Formula**
 - Project duration (**TDEV**) is estimated using:
 - $TDEV = 3 \times (PM)^{(0.33 + 0.2(B - 1.01))}$
 - Where:
 - **TDEV** = Development time (months)
 - **PM** = Effort in person-months
 - **B** = Scale factor exponent
 - The formula predicts the nominal schedule for project completion.
- **Important points about project duration**
 - Depends on project size and complexity.
 - Is **not directly proportional** to the number of employees.
 - Increasing staff does not always reduce development time.
 - Too many people can increase communication overhead and project cost.
- **Characteristics of staffing**
 - Staff requirements vary throughout the project.
 - Different development phases require different numbers of employees.
 - More people do not always mean faster development.

- Rapid increases in team size can cause schedule slippage.
 - **Typical staffing pattern**
 - **Planning Phase:** Small team required.
 - **Development Phase:** Maximum number of staff required.
 - **Testing Phase:** Staff requirement remains high.
 - **Maintenance Phase:** Fewer people are needed.
 - **Factors affecting project duration and staffing**
 - **Project Size:** Larger projects require more effort, time, and staff.
 - **Project Complexity:** Complex systems need skilled personnel and longer schedules.
 - **Personnel Capability:** Experienced developers improve productivity and reduce effort.
 - **Development Environment:** Good CASE tools and modern technologies increase productivity.
 - **Schedule Constraints:** Tight deadlines may require additional staff but increase risk.
 - **Risk Factors:** Technical risks and changing requirements affect staffing and duration.
 - **Relationship between duration and staffing**
 - Project duration and staffing are closely related.
 - Adding people to a delayed project may **not** speed up completion.
 - Larger teams increase communication and coordination overhead.
 - Proper staffing improves productivity and schedule performance.
 - **Advantages of proper duration and staffing estimation**
 - Better resource allocation.
 - Improved project scheduling.
 - Reduced project risk.
 - More accurate cost estimation.
 - Timely project delivery.
-

Agile Planning

- **Agile planning** is an iterative approach where software is developed and delivered in small increments.
- Unlike plan-driven development, the features included in each increment are **not fixed in advance**.
- The content of each increment is decided based on:
 - Current development progress.
 - Customer priorities.
 - Changing requirements.
- Agile planning is **flexible** and can adapt to changing customer needs.
- It accepts that customer priorities and requirements may change during development.
- **Stages of agile planning**
 - **Release planning**
 - Looks ahead for several months.
 - Decides which features should be included in a software release.
 - **Iteration planning**
 - Short-term planning, usually for the next **2–4 weeks**.
 - Focuses on the next increment or iteration of development.
- **Key characteristics**
 - Planning is done incrementally.
 - The whole development team participates in planning.
 - Plans are revised as the project progresses.
 - If problems occur, functionality may be reduced instead of delaying delivery.
- **Advantages**
 - Easily accommodates changing requirements.
 - Delivers working software in small, regular increments.
 - Keeps customer priorities at the center of development.

- Enables faster feedback and continuous improvement.
-

Estimation Techniques

- **Estimation techniques** are methods used to estimate the effort, cost, and time required to develop a software project.
- Organizations use these estimates for project planning and budgeting.
- There are **two main estimation techniques**:
 - **Experience-based estimation**
 - **Algorithmic cost modeling**
- **Experience-based estimation**
 - Based on the project manager's experience with past projects and the application domain.
 - The manager makes an informed judgment about the required effort.
 - Deliverables and software components are identified and estimated individually.
 - Individual estimates are combined to obtain the total effort.
 - Involving multiple people in estimation and discussing their estimates improves accuracy.
- **Problems with experience-based estimation**
 - New projects may be very different from previous ones.
 - Rapid changes in technology can make past experience less useful.
 - Unfamiliar technologies (such as new frameworks or platforms) make accurate estimation difficult.
- **Algorithmic cost modeling**
 - Uses mathematical formulas to estimate project effort.
 - Estimates are based on product size and project/process characteristics.
 - General formula:
 - **Effort = A × Size^B × M**
 - Where:

- **A** = organization-dependent constant.
 - **B** = factor representing increased effort for larger projects.
 - **M** = multiplier based on product, process, and personnel attributes.
- Code size is the most commonly used input for estimation.
- **Estimation accuracy**
 - Exact software size is known only after development is complete.
 - Final size is affected by:
 - Reused components.
 - Programming language.
 - System distribution.
 - Estimates become more accurate as development progresses.
 - Values of factors like **B** and **M** depend on the estimator's judgment.
- **Effectiveness of algorithmic models**
 - Provide a systematic method for effort estimation.
 - Can be complex and difficult to apply.
 - Require estimating many parameters, introducing uncertainty.
 - Commonly used in large organizations, especially in defense and aerospace projects.
- **Key points**
 - Experience-based techniques rely on expert judgment from previous projects.
 - Algorithmic techniques compute effort using mathematical models and estimated project parameters.
 - Both approaches aim to improve software cost and effort estimation.

COCOMO Cost Modelling

- **COCOMO (Constructive Cost Model)** is an **empirical software cost estimation model** used to estimate the effort, cost, and schedule of software projects.

- It is based on data and experience from previous software projects.
- The latest version is **COCOMO II**, which supports modern software development methods and software reuse.
- **Features of COCOMO II**
 - Based on project experience and historical data.
 - Well-documented and independent of any specific software vendor.
 - Considers different development approaches and code reuse.
 - Produces increasingly detailed estimates as more project information becomes available.
- **COCOMO II sub-models**
 - **Application Composition Model**
 - Used when software is built using existing components or prototypes.
 - Based on **Application Points (AP)** and developer productivity.
 - Formula:
 - $PM = (NAP \times (1 - \%Reuse / 100)) / PROD$
 - Where:
 - **PM** = Effort in person-months
 - **NAP** = New Application Points
 - **PROD** = Productivity (Application Points per month)
 - **Early Design Model**
 - Used after requirements are known but before detailed design starts.
 - **Reuse Model**
 - Estimates the effort required to integrate and adapt reusable components.
 - **Post-Architecture Model**
 - Used after the system architecture is completed and more project details are available.
- **Application Composition Model**
 - Suitable for prototyping projects and projects with extensive reuse.

- Takes software tool usage into account.
- Uses developer productivity measured in application points per month.
- **Early Design Model**
 - Used when requirements are available but detailed design has not begun.
 - Formula:
 - $PM = A \times \text{Size}^B \times M$
 - Where:
 - $A = 2.94$
 - **Size** = estimated size in KLOC
 - **B** depends on project characteristics
 - **M** is a multiplier based on cost drivers.
- **Advantages of COCOMO II**
 - Provides a systematic and quantitative estimation method.
 - Supports software reuse and different development styles.
 - Improves planning of effort, cost, and schedule.
 - Estimates become more accurate as project information increases.
- **Limitations**
 - Requires accurate input data for reliable estimates.
 - Many parameters are subjective and depend on expert judgment.
 - More suitable for medium and large projects than very small ones.

Application Point Productivity Chart (COCOMO II)

- **Application Point Productivity Chart in COCOMO II** shows how developer capability and CASE tool maturity affect productivity.
- Productivity is measured in **New Application Points (NAP) per month**.
- As developer experience and tool support improve, productivity increases and the required development effort decreases.
- **Developer capability vs productivity**

- **Very Low Capability** → 4 NAP/Month
- **Low Capability** → 7 NAP/Month
- **Nominal Capability** → 13 NAP/Month
- **High Capability** → 25 NAP/Month
- **Very High Capability** → 50 NAP/Month
- **Very High Capability**
 - Productivity = **50 NAP/Month**.
 - Represents highly skilled developers using advanced CASE tools.
 - Achieves maximum productivity and reduces development effort.
- **Effort calculation in the Application Composition Model**
 - Formula:
 - **PM = NAP / PROD**
 - Where:
 - **PM** = Person-Months
 - **NAP** = New Application Points
 - **PROD** = Productivity (NAP per month)

Developer's Experience and Capability	Very Low	Low	Nominal	High	Very High
ICASE Maturity and Capability	Very Low	Low	Nominal	High	Very High
PROD (NAP/Month)	4	7	13	25	50

- **Example**
 - Given:
 - **NAP = 260**
 - **PROD = 13 NAP/Month (Nominal)**
 - Calculation:
 - **PM = 260 / 13 = 20 Person-Months**

- Therefore, the project requires **20 Person-Months of effort**.
 - **Key points**
 - Higher developer capability → Higher productivity.
 - Better CASE tools → Increased productivity.
 - Higher productivity → Lower person-month effort and faster project completion.
-

Scale Factors in the COCOMO II Post-Architecture Model

- **Scale Factors** in the **COCOMO II Post-Architecture Model** are used to compute the exponent in the effort estimation formula.
- They reflect project characteristics that influence the effort required for software development.
- The five scale factors affect the value of **B** in the equation:
 - $PM = A \times Size^B \times M$
 - Where:
 - **PM** = Effort in Person-Months
 - **A** = Constant (typically 2.94)
 - **Size** = Software size
 - **B** = Exponent computed using scale factors
 - **M** = Effort multiplier
- **The five scale factors are:**
 - **Precedentedness (PREC)**
 - Measures how similar the project is to previous projects.
 - Familiar projects require less effort; new projects require more effort.
 - **Development Flexibility (FLEX)**
 - Measures how much flexibility exists in the development process and requirements.
 - Greater flexibility generally reduces effort.

- **Architecture/Risk Resolution (RESL)**
 - Measures how well the architecture is defined and risks are addressed early.
 - Better risk resolution lowers project uncertainty and effort.
- **Team Cohesion (TEAM)**
 - Measures communication and cooperation among team members.
 - Well-coordinated teams improve productivity.
- **Process Maturity (PMAT)**
 - Measures the maturity and quality of the organization's software development processes.
 - Mature processes reduce development effort and improve efficiency.
- **Example values used in exam problems**
 - Precedentedness = **New Project (4)**
 - Development Flexibility = **Very High (1)**
 - Architecture/Risk Resolution = **Very Low (5)**
 - Team Cohesion = **Nominal (3)**
 - Process Maturity = **Nominal (3)**
 - Constants:
 - $A = 2.94$
 - $\text{Size} = 128$
 - $M = 3.15$
- **Key points**
 - Scale factors determine the exponent **B** in the COCOMO II effort equation.
 - They capture project-specific characteristics beyond code size.
 - Better planning, experienced teams, mature processes, and effective risk management generally reduce development effort.

COCOMO II POST-ARCHITECTURAL MODEL – SCALE FACTORS & EFFORT ESTIMATION

Problem:

Illustrate scale factors used in the exponent computation in the Post-Architectural Model and estimate the effort when the scale factor values are:

- Precedentedness (PREC) = 4
- Development Flexibility (FLEX) = 1
- Architecture / Risk Resolution (RESL) = 5
- Team Cohesion (TEAM) = 3
- Process Maturity (PMAT) = 3

Given:

A = 2.94
Size = 128 KSLOC
M = 3.15

COCOMO II Post-Architectural Model Formulas

Effort equation:

$$PM = A \times (Size)^B \times M$$

Exponent (B) computation:

$$B = 1.01 + \left(\frac{\sum SF}{100} \right)$$

Where:

- PM = Effort in Person-Months
- A = Constant (2.94)
- Size = Software size in KSLOC
- B = Exponent derived from scale factors
- M = Effort Multiplier
- SF = Scale Factors

Summary of Inputs:

- Sum of Scale Factors ($\sum SF$) = $4 + 1 + 5 + 3 + 3 = 16$
- A (Constant) = 2.94
- Size = 128 KSLOC
- M (Effort Multiplier) = 3.15

Step 1: Sum of Scale Factors

$$\sum SF = 4 + 1 + 5 + 3 + 3$$

$$\sum SF = 16$$

Step 2: Compute Exponent (B)

$$B = 1.01 + \left(\frac{\sum SF}{100} \right)$$

$$B = 1.01 + \left(\frac{16}{100} \right)$$

$$B = 1.01 + 0.16$$

$$B = 1.17$$

Step 3: Size Calculation

$$Size^B = 128^{1.17}$$

$$128^{1.17} \approx 292.53$$

Step 4: Effort Calculation

$$PM = A \times (Size)^B \times M$$

$$PM = 2.94 \times 292.53 \times 3.15$$

$$PM = 860.04 \times 3.15$$

$$PM \approx 2708 \text{ person-months}$$

Final Answer

Estimated effort required to develop the project is approximately:

2708 Person-Months

Design Patterns

- **Design patterns** are reusable solutions to commonly occurring software design problems.
- They represent a way of reusing abstract knowledge about a problem and its solution.
- A design pattern describes the problem and the essence of its solution.
- It should be general enough to be reused in different situations.
- Pattern descriptions often use object-oriented concepts such as **inheritance** and **polymorphism**.
- **Purpose of design patterns**
 - Capture best practices in software design.
 - Promote good design solutions.
 - Allow developers to reuse proven design experience.
 - Improve software maintainability and consistency.
- **Main elements of a design pattern**
 - **Name** – A meaningful identifier for the pattern.
 - **Problem description** – Explains the design problem the pattern solves.
 - **Solution description** – Provides a general template for solving the problem (not a complete design).
 - **Consequences** – Describes the results, benefits, and trade-offs of applying the pattern.
- **Observer Pattern (Example)**
 - Separates the display of an object's state from the object itself.
 - Allows multiple views or displays of the same data.
 - When the object's state changes, all displays are automatically notified and updated.
 - Useful when multiple representations (such as graphical and tabular views) of the same information are required.
 - The object maintaining the data does not need to know the specific display formats being used.
 - One limitation is that optimizing display performance can be difficult.

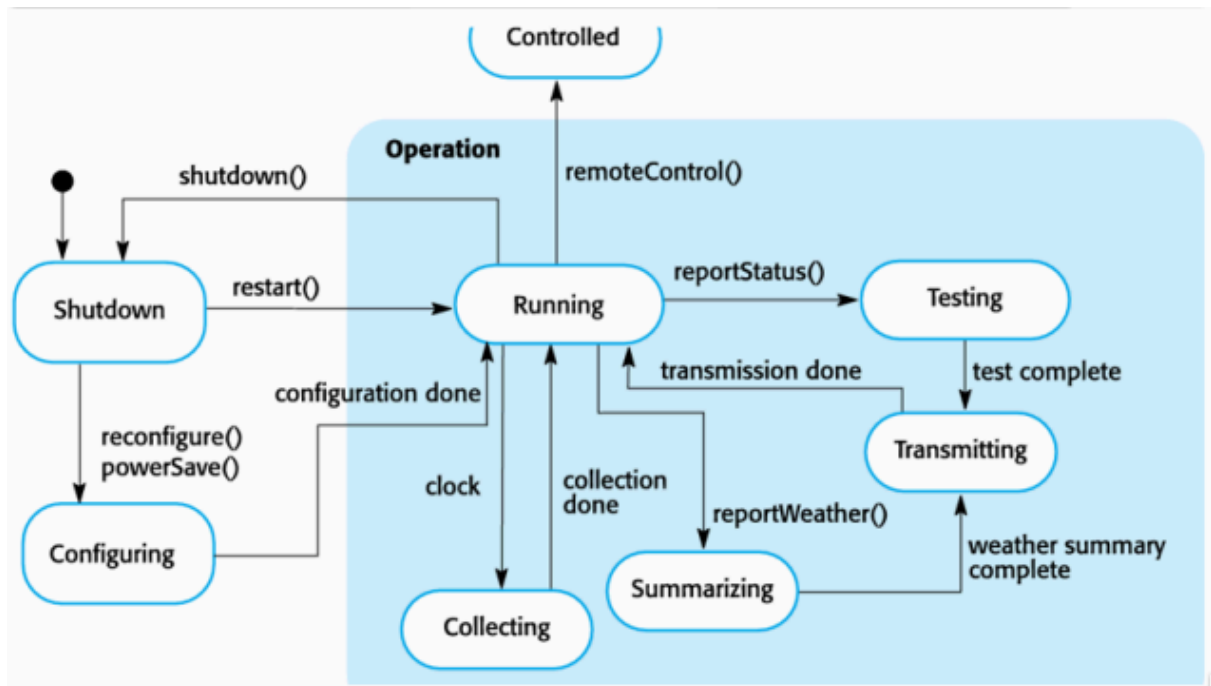
State Model for Weather Report System

- A **State Model (State Machine Diagram)** shows how a system changes from one state to another when events occur.
- It represents the **dynamic behavior** of a system and the **state transitions** triggered by operations or events.
- State models are useful for systems that respond to external events during execution.
- **States in the Weather Report System**
 - **Shutdown**
 - Initial state of the system.
 - The weather station is not operating.
 - **Configuring**
 - System configuration takes place.
 - Operations such as `reconfigure()` and `powerSave()` are performed.
 - After configuration, the system moves to the **Running** state.
 - **Running**
 - Main operating state of the weather station.
 - Can be controlled remotely using `remoteControl()`.
 - **Collecting**
 - Collects weather data (temperature, pressure, humidity, etc.) from sensors when a clock signal occurs.
 - **Summarizing**
 - Processes and summarizes the collected weather data.
 - **Transmitting**
 - Sends the weather report using the `reportWeather()` operation after summarization.
 - **Testing**
 - Performs system testing using `reportStatus()`.

- After testing is complete, the system returns to the **Transmitting** state.

- **Typical state transitions**

- Shutdown → Configuring
- Configuring → Running
- Running → Collecting
- Collecting → Summarizing
- Summarizing → Transmitting
- Transmitting → Testing
- Testing → Transmitting
- The system may return to Shutdown when powered off.



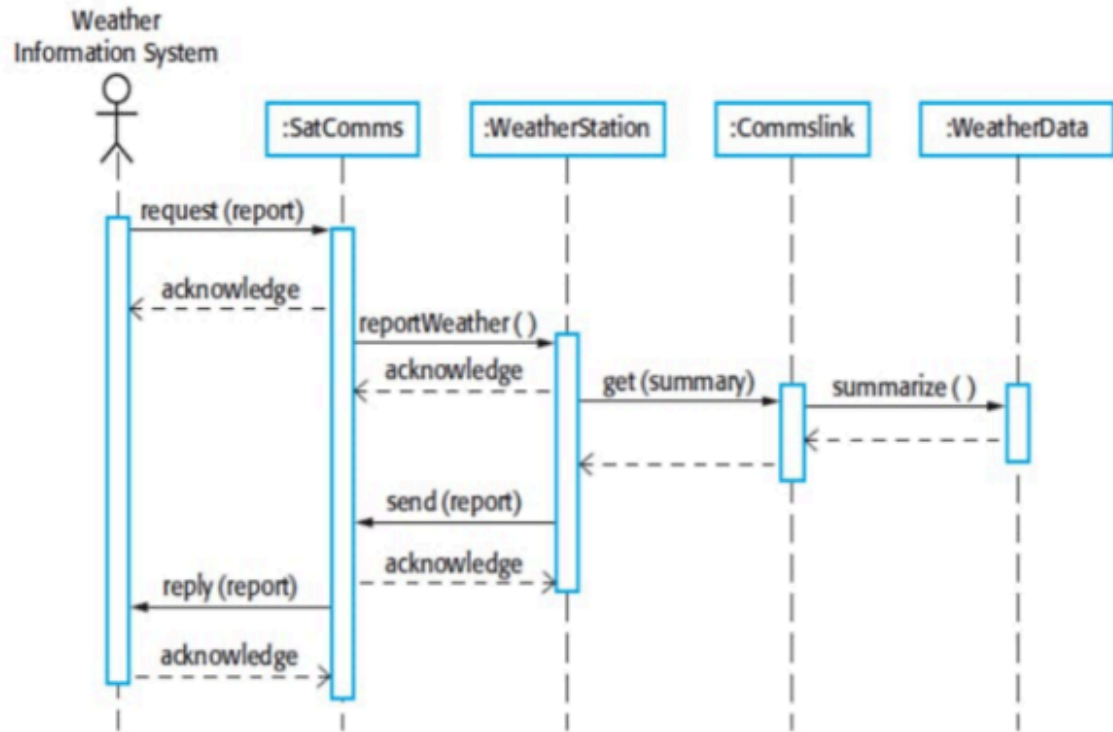
- **Purpose of the state model**

- Shows how the weather station behaves during operation.
- Describes the sequence of states involved in collecting, processing, testing, and transmitting weather data.
- Helps developers understand the system's runtime behavior and event-driven transitions.

Sequence Diagram for Weather Station Data Collection

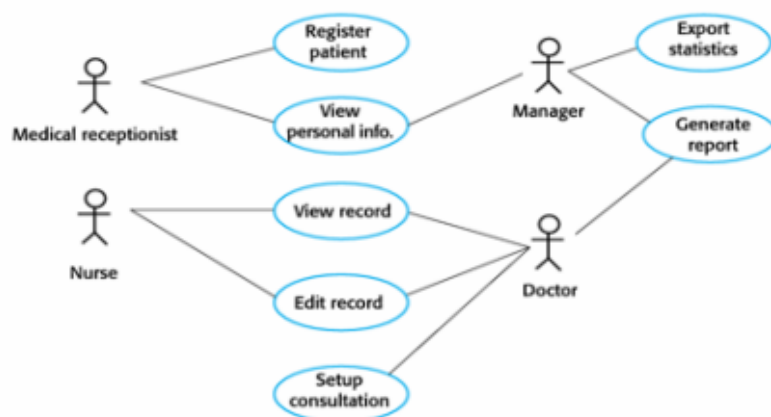
- A **Sequence Diagram** is a UML interaction diagram that shows the sequence of messages exchanged between objects over time.
- Objects are arranged horizontally, and **time progresses from top to bottom**.
- Interactions between objects are represented using arrows.
- **Objects involved in the Weather Station sequence diagram**
 - Weather Information System (Actor)
 - SatComms (Satellite Communication System)
 - WeatherStation
 - Commslink
 - WeatherData
- **Sequence of interactions**
 1. Weather Information System sends a **report** request to **SatComms** to collect weather data.
 2. **SatComms** sends an **acknowledgement** to the Weather Information System.
 3. **SatComms** sends **reportWeather()** to **WeatherStation**.
 4. **WeatherStation** sends an **acknowledgement** to **SatComms**.
 5. **WeatherStation** sends a **get(summary)** request to **Commslink**.
 6. **Commslink** sends a **summarize()** request to **WeatherData**.
 7. **WeatherData** processes the sensor data and returns the **summarized weather information**.
 8. **WeatherStation** sends **send(report)** to **SatComms**.
 9. **SatComms** sends **reply(report)** to the **Weather Information System**.
 10. Weather Information System sends a final **acknowledgement**.
- **Purpose of the sequence diagram**
 - Shows how weather data is **requested, collected, processed, summarized, and transmitted**.

- Illustrates the order of communication between system components.
- Helps understand the dynamic interaction among the objects in the weather station system.



Mental Health Care Use Case Diagram

- A **Use Case Diagram** is a UML diagram that shows the interactions between **actors** (users) and the **system functions (use cases)**.
- It helps identify what services the system provides and who can access them.



Use cases for Mentcare System

- **Actors**
 - Medical Receptionist
 - Nurse
 - Doctor
 - Manager
- **Main use cases**
 - Register Patient
 - View Personal Information
 - View Record
 - Edit Record
 - Setup Consultation
 - Export Statistics
 - Generate Report
- **Register Patient**
 - Performed by the **Medical Receptionist**.
 - Registers new patients and stores their details in the system.
- **View Personal Information**
 - Performed by the **Medical Receptionist** and **Manager**.
 - Allows viewing of patient personal details when required.
- **View Record**
 - Performed by **Doctors** and **Nurses**.
 - Allows access to patient medical records for treatment and monitoring.
- **Edit Record**
 - Performed by **Doctors** and **Nurses**.
 - Used to update diagnosis, treatment details, and observations in patient records.
- **Setup Consultation**
 - Performed by the **Doctor**.

- Used to schedule and manage patient consultations.
 - **Export Statistics**
 - Performed by the **Manager**.
 - Exports statistical information related to patient care and clinic performance.
 - **Generate Report**
 - Performed by the **Manager** and **Doctor**.
 - Generates reports for administration and treatment purposes.
 - **Actor–Use Case Mapping**
 - **Medical Receptionist:** Register Patient, View Personal Information
 - **Nurse:** View Record, Edit Record
 - **Doctor:** View Record, Edit Record, Setup Consultation, Generate Report
 - **Manager:** View Personal Information, Export Statistics, Generate Report
-

Activity Bar Chart

- **Activity Bar Chart (Gantt Chart)** is a graphical representation of project tasks against time.
- It shows:
 - Duration of each task.
 - Start and finish times.
 - Task dependencies.
 - Project milestones.
- It helps managers plan, monitor, and track project progress.
- **Data required to construct an Activity Bar Chart**
 - List of project tasks/activities.
 - Duration of each task.
 - Dependencies between tasks (which task must finish before another starts).
 - Milestones to indicate important project events.

- Resource allocation information (optional).

- **Procedure to construct the chart**

1. Identify all project activities from the requirements.
2. Determine dependencies between activities.
3. Estimate the duration of each activity.
4. Allocate people and resources to activities.
5. Draw horizontal bars representing task durations on a timeline.
6. Mark milestones at important completion points.
7. Show dependencies using connecting arrows or by scheduling dependent tasks after their predecessors.

- **Example**

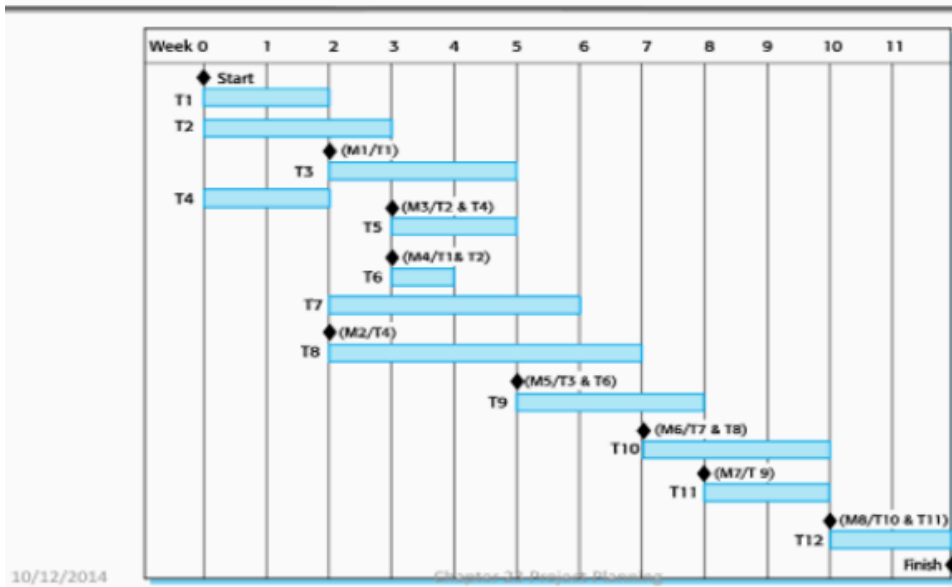
Task	Effort (person-days)	Duration (days)	Dependencies
T1	15	10	
T2	8	15	
T3	20	15	T1 (M1)
T4	5	10	
T5	5	10	T2, T4 (M3)
T6	10	5	T1, T2 (M4)
T7	25	20	T1 (M1)
T8	75	25	T4 (M2)
T9	10	15	T3, T6 (M5)
T10	20	15	T7, T8 (M6)
T11	10	10	T9 (M7)
T12	20	10	T10, T11 (M8)

10/12/2014

Chapter 23 Project Planning

31

Activity bar chart



- **Milestones**
 - Requirements Approved
 - Design Completed
 - Coding Completed
 - Testing Completed
 - Project Delivered
- **Advantages**
 - Clearly shows task durations and schedules.
 - Makes dependencies easy to identify.
 - Helps monitor project progress.
 - Improves planning and resource allocation.

Use Case Description for Weather Forecasting Station

- **System:** Weather Station
- **Use Case:** Report Weather
- **Actors:** Weather Information System, Weather Station
- **Description:**

- The weather station sends a summary of the weather data collected from its instruments to the weather information system.
 - The data includes:
 - Maximum, minimum, and average ground temperatures.
 - Maximum, minimum, and average air pressures.
 - Maximum, minimum, and average wind speeds.
 - Total rainfall.
 - Wind direction sampled at five-minute intervals.
 - **Stimulus:**
 - The weather information system establishes a satellite communication link with the weather station and requests transmission of the weather data.
 - **Response:**
 - The summarized weather data is transmitted to the weather information system.
 - **Comments:**
 - Weather stations are usually configured to report **once every hour**.
 - The reporting frequency may vary between stations and can be modified in the future.
-